

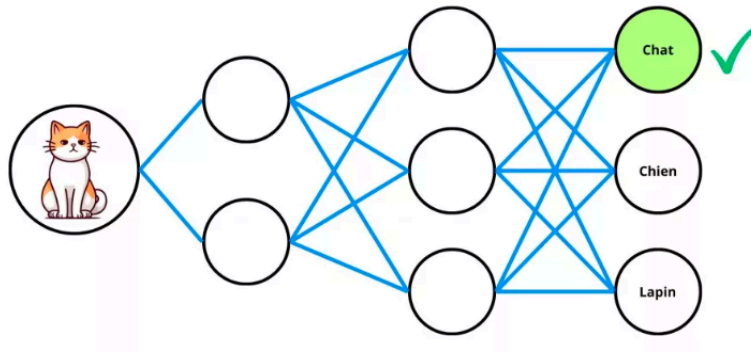
UNIVERSITÉ MOULOUD MAMMERI DE TIZI-OUZOU

FACULTÉ GÉNIE ÉLECTRIQUE INFORMATIQUE
DÉPARTEMENT D'INFORMATIQUE



Projet Deep Learning

GENERATIVE ADVERSARIAL NETWORK



REBHI Idir & LARBI Siham

30/04/2025

MASTER 1 – S2I

Introduction

Les Generative Adversarial Networks, ou **GANs**, ont profondément marqué le domaine de l'intelligence artificielle en permettant la génération automatique de données réalistes. Introduits en 2014 par Ian Goodfellow, ces réseaux se basent sur une idée originale : mettre en compétition deux modèles, un générateur et un discriminateur, dans un processus d'apprentissage mutuel. Le générateur tente de produire des données artificielles proches de la réalité, tandis que le discriminateur cherche à faire la distinction entre les vraies données et celles générées.

Ce mécanisme d'entraînement compétitif a permis des avancées spectaculaires dans la génération d'images, de vidéos, de musiques et bien plus encore. Yann LeCun, figure majeure de l'IA, a d'ailleurs qualifié les GANs de “plus belle idée en intelligence artificielle des dix dernières années”.

L'intérêt pour ces modèles ne cesse de croître, notamment grâce à leur capacité à produire des résultats visuellement impressionnants. Que ce soit dans la création d'avatars réalistes, la restauration d'images endommagées ou encore la génération de visages fictifs, les GANs ouvrent la voie à de nombreuses applications innovantes qui repoussent les limites de l'intelligence artificielle.

Ce rapport se propose d'explorer le fonctionnement des GANs, leur mise en œuvre pratique ainsi que les résultats obtenus à partir d'un jeu de données.

Sommaire :

Introduction

- Présentation et importance des GANs.
- Objectif du rapport et méthodologie utilisée.

I. Présentation des GANs.....3

II. Les GANs : Comment ça marche ?.....5

- Explication du rôle du générateur et du discriminateur ainsi que de leur interaction.
- Fonction de valeur (loss function) et son interprétation.
- Identification du problème initial de convergence et solution proposée pour y remédier.

III. Limitations des GANs.....9

- Principales limitations : instabilité de l'entraînement, sensibilité aux hyperparamètres, besoins en ressources et manque d'interprétabilité.
- Problématique du mode collapse et ses conséquences.

IV. GAN classique vs autres architectures.....11

- Comparaison entre GANs classiques, DCGAN, StyleGAN et CGAN, avec leurs spécificités respectives.

V. Les DCGANs11

- Introduction aux DCGANs et leurs avantages pour la génération d'images.
- Bonnes pratiques et avantages des DCGANs par rapport aux autres architectures.
- Défis rencontrés dans l'évaluation des résultats générés.

VI. Implémentation.....16

VII. Conclusion.....23

VIII. Bibliographie.....24

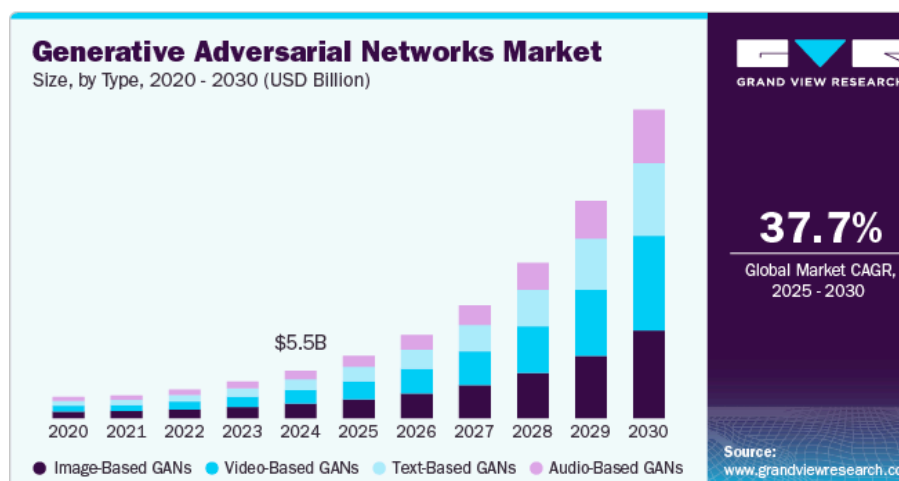
I.Présentation des GANs :

Les Generative Adversarial Networks (GANs), proposés en 2014 par Ian Goodfellow et ses collaborateurs, sont une classe de modèles de génération de données basés sur un apprentissage non supervisé. Leur principe repose sur un jeu à somme nulle (théorie des jeux - minmax) entre deux réseaux de neurones : un **générateur**, qui cherche à créer des données similaires à celles du jeu d'entraînement, et un **discriminateur**, chargé de distinguer les données réelles de celles produites artificiellement. Le système apprend en opposant ces deux réseaux dans une boucle d'entraînement continue, jusqu'à ce que le générateur parvienne à tromper le discriminateur. Cette architecture a permis d'importantes avancées dans la génération d'images, de sons et d'autres types de données complexes (Goodfellow et al., 2014)

Les GANs sont un cadre d'apprentissage pour les modèles génératifs, où le problème est formulé comme un **jeu à deux joueurs** entre un générateur, qui produit des exemples, et un discriminateur, qui les évalue. Cette approche a montré de bons résultats dans des tâches telles que la synthèse d'images ou l'augmentation de données. (Creswell et al. 2018).

Quelques statistiques sur l'IA générative et les GANs:

Selon une étude de marché, le secteur des GANs devrait atteindre 36,01 milliards USD d'ici 2030, avec un taux de croissance annuel de 37,7 % (Grand View Research, 2024).



(source : GRAND VIEW RESEARCH)

En 2023, les réseaux antagonistes génératifs (GANs) représentaient plus de **74 % du marché des modèles d'IA générative**, surpassant nettement les architectures de type Transformer (AmplifAI, 2024).

II. Les GANs comment ça marche ? :

- Dans le cas qui suit, il s'agira des GANs générant des chiffres manuscrits.
Toutefois, l'explication peut être étendue à d'autres types de génération.

Soient les variables et fonctions suivantes :

- x : une image réelle,
- $D(x)$: la probabilité que l'image soit réelle ou générée,
- z : un vecteur latent (bruit aléatoire),
- $G(z)$: image générée par le générateur à partir de z (soit une image fausse).

Générateur : Il essaie de générer des images fausses à partir de z pour tromper le discriminateur. Autrement dit, il cherche à maximiser la probabilité suivante : $D(G(z))$, c'est-à-dire que le discriminateur prenne $G(z)$ pour une image réelle.

Discriminateur : Il cherche à éviter de se faire tromper par le générateur. Plus précisément, il devient de plus en plus strict et souhaite minimiser $D(G(z))$, c'est-à-dire que $G(z)$ soit perçu comme une image fausse.

En résumé :

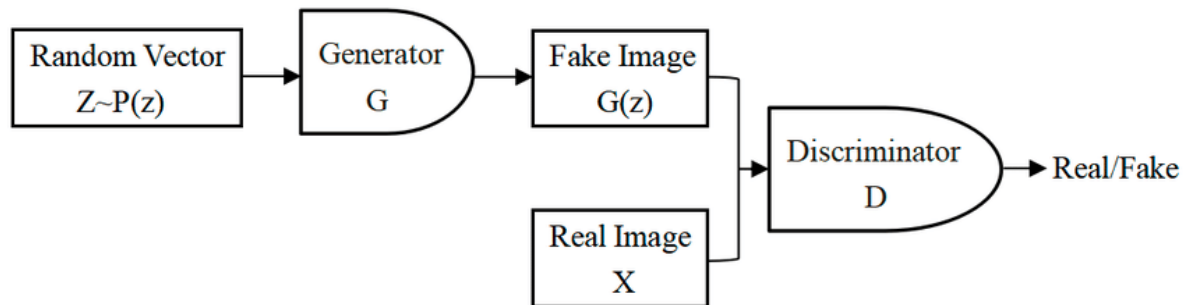
- $D(x)$ doit être élevé si x est une image réelle,
- $D(G(z))$ doit être faible si $G(z)$ est une image générée.

Le générateur cherche à maximiser $D(G(z))$ pour que ses images soient prises pour des images réelles.

Le **discriminateur** prend en entrée une image de taille **64x64x1** et en sort une probabilité (comprise entre 0 et 1) indiquant si l'image est réelle ou générée.

Le **générateur**, quant à lui, cherche à tromper le discriminateur en générant des images qui semblent réelles.

Ceci est illustré dans la figure suivante :



(source : Spectra-GANs, 2025)

Le GAN comme un jeu minimax :

Le GAN peut être vu comme un jeu de type minimax où :

- Le **discriminateur (D)** veut maximiser ses chances de correctement classer les images,
- Le **générateur (G)** veut minimiser les chances que **D** découvre que les images sont fausses.

Avec la fonction de valeur suivante :

$$V(D, G) = E_{x \sim p_{data}(x)}[\log D(x)] + E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

Détail des deux termes :

1. $E_{x \sim p_{data}(x)}[\log D(x)] :$

- On prend une vraie image x issue des données réelles.
- On souhaite que $D(x)$ soit proche de 1, donc que le discriminateur reconnaisse l'image comme vraie.
- D veut maximiser ce terme pour bien classer les vraies images.

2. $E_{z \sim p_z(z)}[\log(1 - D(G(z)))] :$

- On génère une image avec du bruit z , via $G(z)$
- On demande à D de reconnaître que ce n'est pas une vraie image, donc $D(G(z))$ doit être proche de 0.
- D veut maximiser ce terme pour rejeter les fausses images.
- Mais G veut minimiser ce terme pour que ses fausses images soient prises pour des vraies (c'est-à-dire faire croire à D que $G(z)$ est réel).

Exemple :

On suppose que :

- Le discriminateur D donne une probabilité entre 0 et 1, $D \in [0, 1]$.
- On a une image réelle notée x : D pense que c'est une vraie image avec 80% de certitude, donc $D(x) = 0.8$.
- Le générateur G produit une image artificielle à partir d'un bruit z , notée $G(z)$ et D estime qu'elle est vraie à 20%, donc $D(G(z)) = 0.2$

Calcul de la fonction objectif :

$$V(D, G) = \log(D(x)) + \log(1 - D(G(z)))$$

$$V(D, G) = \log(0.8) + \log(1 - 0.2) = \log(0.8) + \log(0.8)$$

$$V(D, G) = -0.2231 + (-0.2231) = -0.4462$$

Le discriminateur (D) essaie de *maximiser* la fonction de valeur $V(D,G)$. Il veut que $D(x)$ soit proche de 1 (identifier correctement les données réelles) et que $D(G(z))$ soit proche de 0 (identifier correctement les données générées comme fausses, donc $1-D(G(z))$ proche de 1). Maximiser le logarithme de ces valeurs positives maximise $V(D,G)$.

Le générateur (G) essaie de *minimiser* la fonction de valeur $V(D,G)$. Il veut que $D(G(z))$ soit proche de 1 (tromper le discriminateur en lui faisant croire que les données générées sont réelles, donc $1-D(G(z))$ proche de 0). Minimiser le logarithme de cette dernière quantité minimise $V(D,G)$.

Dans notre exemple, la valeur de $V(D,G) = -0.4462$ reflète une situation où le discriminateur est plutôt performant

Une valeur plus élevée de $V(D, G)$ indique que le discriminateur distingue efficacement les données réelles des données générées, tandis qu'une valeur plus faible suggère que le générateur parvient à mieux tromper le discriminateur. Le tableau suivant illustre ceci :

Image réelle	Image générée	$D(x)$	$D(G(z))$	$V(D,G)$
Oui	Oui	0.95	0.05	≈ -0.1
Oui	Oui	0.8	0.4	≈ -0.73
Oui	Oui	0.6	0.4	≈ -1.02
Incertain	Incertain	0.5	0.5	≈ -1.38

→ Le discriminateur a produit une sortie 0,5 pour une image générée et une image réelle, donc il n'arrive plus à distinguer les vraies images des fausses.

→ Plus on descend dans le tableau, plus le générateur **progress**e, et le discriminateur devient **incertain**

Problème potentiel :

Selon **Ian Goodfellow**, La fonction d'origine pour le générateur :

$\min V(D, G) = E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$, pose **un problème** au début de l'entraînement.

Au début, le générateur G produit des images **très mauvaises**. Donc, $D(G(z))$ est proche de **0**, car le discriminateur détecte facilement que ce sont des fakes.

$$\log(1 - D(G(z))) \approx \log(1 - 0) = \log(1) = 0$$

Mais si $D(G(z))$ est **trop proche de 0**, la dérivée (le **gradient**) devient **très faible**. Et donc : **le générateur ne reçoit presque aucun signal d'apprentissage** → il a du mal à s'améliorer.

Solution proposée par Goodfellow :

À la place, on entraîne le générateur à maximiser $\log(D(G(z)))$, cette fonction donne des meilleurs gradients quand $D(G(z))$ est proche de 0.

Exemple :

Si $D(G(z)) = 0.01$ alors :

$$\log(1 - 0.01) \approx -0.01 \text{ (gradient faible)}$$

$$\log(0.01) \approx -4.6 \text{ (gradient fort)}$$

III.Limitations des GANs :

1. Instabilité de l'entraînement

L'entraînement des GANs est souvent instable en raison de la dynamique compétitive entre le générateur et le discriminateur. Cette instabilité peut entraîner des oscillations, une non-convergence ou des gradients nuls, rendant le processus d'optimisation délicat. (Goodfellow et al., 2014).

2. Sensibilité aux hyperparamètres

Les performances des GANs dépendent fortement du choix des hyperparamètres. Une mauvaise configuration peut aggraver les problèmes d'instabilité et de surapprentissage,

nécessitant souvent un réglage fin et empirique (Saxena & Cao, 2020).

3. Besoins élevés en données et en puissance de calcul

Les GANs exigent souvent de grandes quantités de données pour produire des résultats réalistes et diversifiés. De plus, leur entraînement est intensif sur le plan computationnel, ce qui peut limiter leur accessibilité dans des environnements à faibles ressources (Clickworker, 2023).

4. Manque d'interprétabilité

Les GANs fonctionnent comme des « boîtes noires » : il est difficile de comprendre comment et pourquoi ils produisent certains résultats, ce qui limite leur utilisation dans des contextes nécessitant de la transparence (Mustafa, 2022).

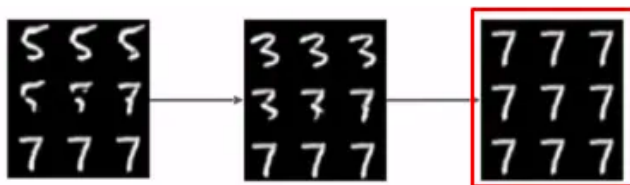
→ Il y'a aussi le problème du **mode collapse**,

une situation où le générateur G ne produit qu'un nombre très limité de types d'images, voire une seule image, même si les données réelles sont très variées.

Cela signifie que G "trouve quelque chose" qui trompe efficacement le discriminateur D, et il ne se donne plus la peine de créer de la diversité dans ses sorties.

Pourquoi cela se produit ? Lorsqu'une certaine sortie $G(z)$ trompe efficacement D, le générateur est fortement récompensé (par sa fonction de perte) et apprend à reproduire cette sortie encore et encore, peu importe le bruit z d'entrée. Il ignore donc d'autres régions de l'espace de données réelles.

On peut observer ceci dans l'illustration suivante (Génération de nombres manuscrits) :



(source : <https://medium.com/@Mustafa77/gans-specialization-part6-791e4e1358c5>)

IV. GAN classique vs d'autres architectures :

Type de GAN	Caractéristiques	Avantages	Limites
GAN	GAN est le model de base avec MLP ; Jeu minimax	Simple	Instable, mode collapse
DCGAN	GAN avec CNNs au lieu de MLP	Stable sur MNIST	Pas adapté à des résolutions plus hautes
StyleGAN	progressive growing	Images ultra-réalistes	Très couteux en ressources
CGAN	Ajout d'une condition (étiquette) à G et D	Génération contrôlé	Moins performant si la condition est mal utilisée

V. Les DCGANs :

→Le but de ce travail est d'implémenter enfin, un modèle qui va générer des images, les DCGANs est l'architecture choisie.

Face aux limitations des GANs classiques, plusieurs variantes ont été proposées pour améliorer la stabilité, la qualité des images générées, et la robustesse de l'apprentissage. Parmi elles, les Deep Convolutional GANs (DCGANs) représentent une avancée majeure, en particulier dans le traitement d'images.

Les GANs originaux utilisent des réseaux de neurones fully-connected (densément connectés), peu adaptés aux données visuelles, car ils ne capturent pas efficacement les relations spatiales entre les pixels. De plus, les GANs souffrent souvent d'une instabilité d'entraînement, d'un mode collapse et d'une sensibilité excessive aux hyperparamètres.

Pour répondre à ces défis, Radford et al. (2015) ont proposé les DCGANs, une architecture qui remplace les couches denses par des réseaux convolutifs profonds dans le générateur et le discriminateur. Cela permet de tirer parti des techniques de traitement d'images pour améliorer la

qualité et la diversité des images générées, tout en rendant l'entraînement plus stable.

Architecture du DCGAN

a. Générateur (G)

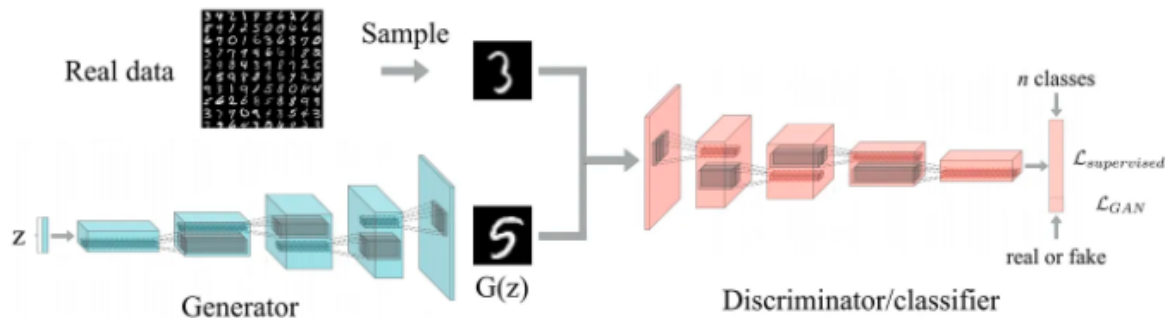
Le générateur prend en entrée un vecteur de bruit z , typiquement de taille 100, tiré d'une distribution normale ou uniforme. Il le transforme en image via une série de **convolutions transposées** (aussi appelées "deconvolutions").

- **Upsampling progressif** : chaque couche augmente la taille de l'image.
- **Fonctions d'activation ReLU** dans toutes les couches sauf la dernière, où **tanh** est utilisée pour normaliser les pixels dans l'intervalle $[-1, 1]$.
- **Batch Normalization** après chaque couche pour stabiliser et accélérer l'apprentissage.

b. Discriminateur (D)

Le discriminateur reçoit une image (réelle ou générée) et tente de prédire si elle est authentique.

- Utilise des **couches de convolution** avec **stride** > 1 pour réduire progressivement la taille de l'image (downsampling).
- **LeakyReLU** comme fonction d'activation pour éviter les gradients nuls.
- **Batch Normalization** appliquée sauf dans la première couche, pour améliorer la généralisation.
- Pas de **pooling** : les DCGANs évitent les couches MaxPooling/AvgPooling pour un meilleur contrôle spatial.



(source : <https://medium.com/the-ai-team>)

Bonnes pratiques pour stabiliser l'apprentissage :

- **Suppression des couches de pooling** au profit de convolutions avec stride.
- **Utilisation systématique de Batch Normalization**, sauf dans la couche de sortie du générateur et dans la première couche du discriminateur.
- **Remplacement de ReLU par LeakyReLU** dans le discriminateur.
- **Initialisation des poids** selon une loi normale $N(0, 0.02)$ pour faciliter la convergence.
- Utilisation de l'**optimiseur Adam**, avec des hyperparamètres ajustés : learning rate = 0.0002 ...

Avantages des DCGANs :

- **Stabilité accrue de l'entraînement** : les problèmes d'explosion ou disparition du gradient sont réduits.
- **Meilleure qualité visuelle des images** : textures plus nettes, détails plus fins.
- **Apprentissage non supervisé efficace** : le discriminateur apprend une représentation

utile même sans étiquettes.

- **Moins de cas de mode collapse**, même si le problème n'est pas totalement résolu.

Comment se déroule l'entraînement d'un DCGAN ? :

Le générateur s'améliore grâce au signal d'erreur qu'il reçoit du discriminateur via le processus de rétropropagation des gradients :

Le générateur prend un bruit aléatoire et le transforme en une image synthétique à travers ses couches.

Évaluation par le Discriminateur : Cette image générée est présentée au discriminateur. Le discriminateur produit une sortie indiquant sa conviction que l'image est réelle ou fausse.

Calcul de la Perte du Générateur : La perte du générateur est calculée en comparant la sortie du discriminateur pour l'image générée à la cible "réel". Si le discriminateur pense que l'image est fausse (sortie faible), la perte du générateur sera élevée.

Rétropropagation des Gradients : La perte du générateur est ensuite utilisée pour calculer les gradients des poids de toutes les couches du générateur qui ont contribué à la création de cette image. C'est la règle de la chaîne du calcul différentiel qui permet de propager l'erreur à travers le réseau.

→ L'idée est de déterminer comment chaque poids dans le générateur a influencé la sortie finale (l'image générée) et donc la perte.

Mise à Jour des Poids du Générateur : L'optimiseur du générateur (Adam) utilise ces gradients pour ajuster les poids du générateur. L'objectif de cet ajustement est de modifier les poids de manière à ce que la prochaine image générée (à partir d'un bruit similaire) soit plus susceptible de "tromper" le discriminateur (c'est-à-dire, de produire une sortie plus proche de "réel").

Apprentissage de Caractéristiques : Au fil du temps, les noyaux (filtres) dans les couches Conv2DTranspose du générateur apprennent à synthétiser des caractéristiques de plus en plus complexes qui ressemblent aux caractéristiques présentes dans les images réelles (par exemple, des courbes, des coins, des textures spécifiques aux chiffres). Les premières images générées

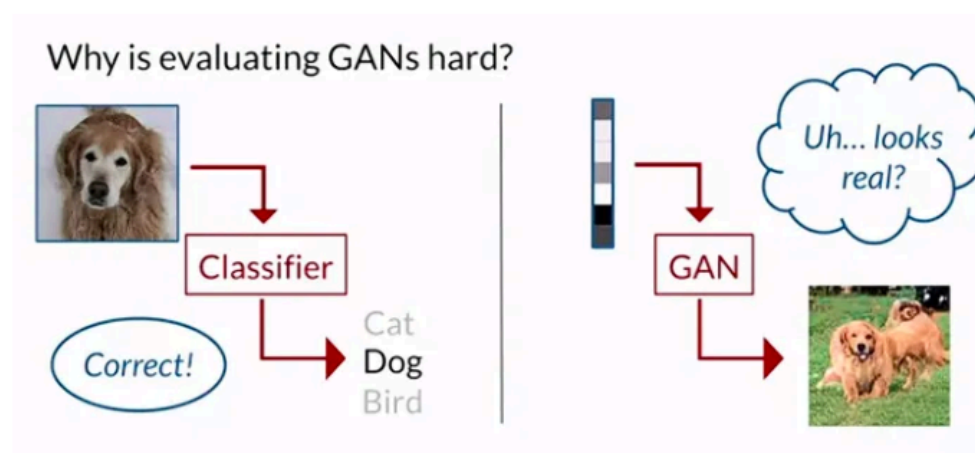
sont généralement du bruit informe. À mesure que l'entraînement progresse, le générateur apprend à structurer ce bruit en des formes reconnaissables. Les couches Conv2DTranspose avec des upsampling progressifs permettent de construire l'image à différentes échelles, en ajoutant des détails de plus en plus fins.

Le générateur apprend en "essayant" de produire des images et en voyant comment le discriminateur réagit. Si le discriminateur détecte facilement une caractéristique comme étant fausse, le gradient rétropropagé informera le générateur de modifier ses poids pour éviter de générer cette caractéristique de la même manière à l'avenir et d'essayer autre chose.

Comment évaluer un DCGAN ? :

Évaluer un modèle DCGAN est intrinsèquement plus complexe qu'évaluer un modèle de classification ou de régression supervisée. En effet, l'objectif d'un DCGAN ou GAN est de générer des données qui ressemblent à la distribution des données réelles, et il n'y a pas de métrique unique et parfaite pour mesurer cela. La matrice de confusion par exemple utilisée dans l'évaluation de modèles de classification est difficilement applicable ici.

L'évaluation des GANs représente un défi majeur et reste un domaine de recherche très actif, ayant récemment connu d'importants progrès. Par exemple, dans les tâches de classification, la présence d'étiquettes permet d'évaluer les modèles sur des bases objectives, comme un ensemble de test séparé. En revanche, dans le cas des GANs, l'absence de telles références rend l'évaluation beaucoup plus complexe (Mustafa, 2020).



(source : <https://medium.com/@Mustafa77/gans-specialization-part5-b92eee81f42e>)

→ Cela dit, on peut toujours se fier à notre perception visuelle pour évaluer les résultats. La recherche sur les GANs est particulièrement active en ce et évolue à une vitesse remarquable. Ainsi, malgré les défis, plusieurs techniques d'évaluation existent, mais elles ne seront pas abordées dans le cadre de ce rapport.

→ Pour aller plus loin ce lien est intéressant : [Cliquez ici](#)

VI.Implémentation :

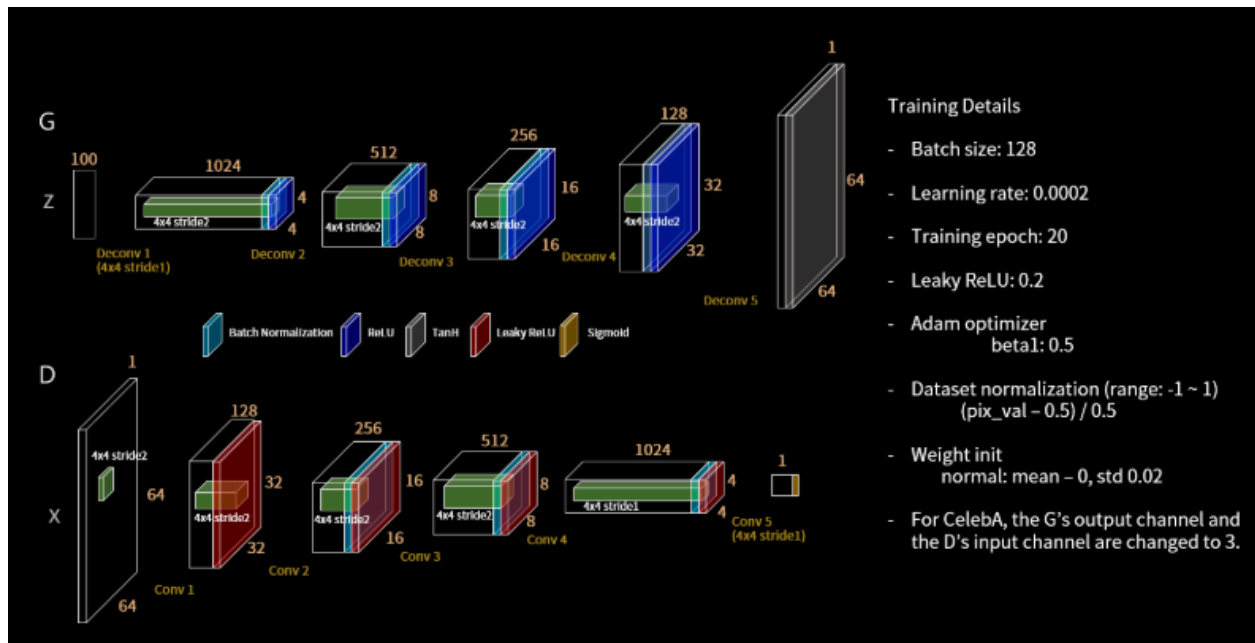
Génération de chiffres avec GAN et DCGAN par (znxlwmHyeonwoo Kang):

→ [Lien pour accéder à cette implémentation](#)

GAN



DCGAN

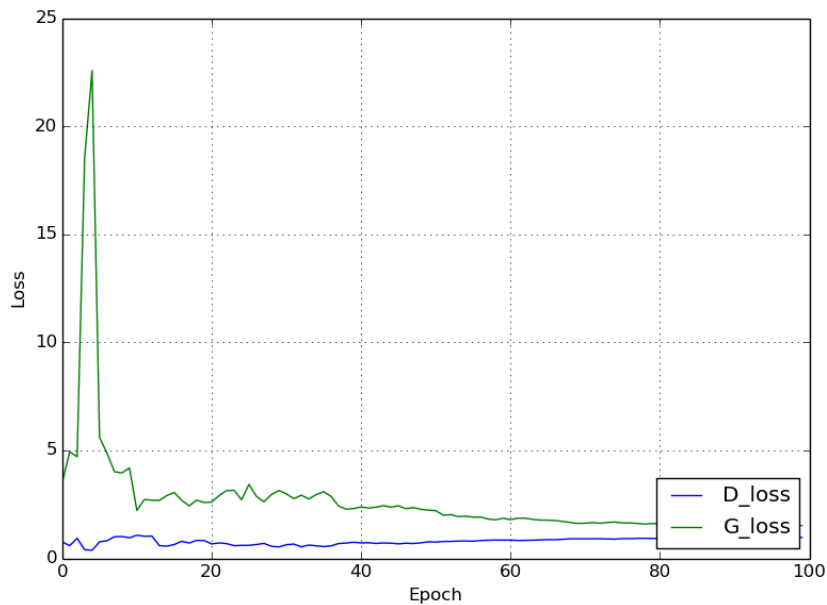


• MNIST vs Generated images

MNIST	GAN after 100 epochs	DCGAN after 20 epochs
	Epoch 100	Epoch 20

Les résultats obtenues à l'époch 20 par le DCGAN, si on considère la perception humaine comme métrique d'évaluation, les résultats sont impressionnants.

Evolution de la courbe de perte :



Le modèle converge, le discriminateur n'est plus en mesure de distinguer les vraies des fausses images

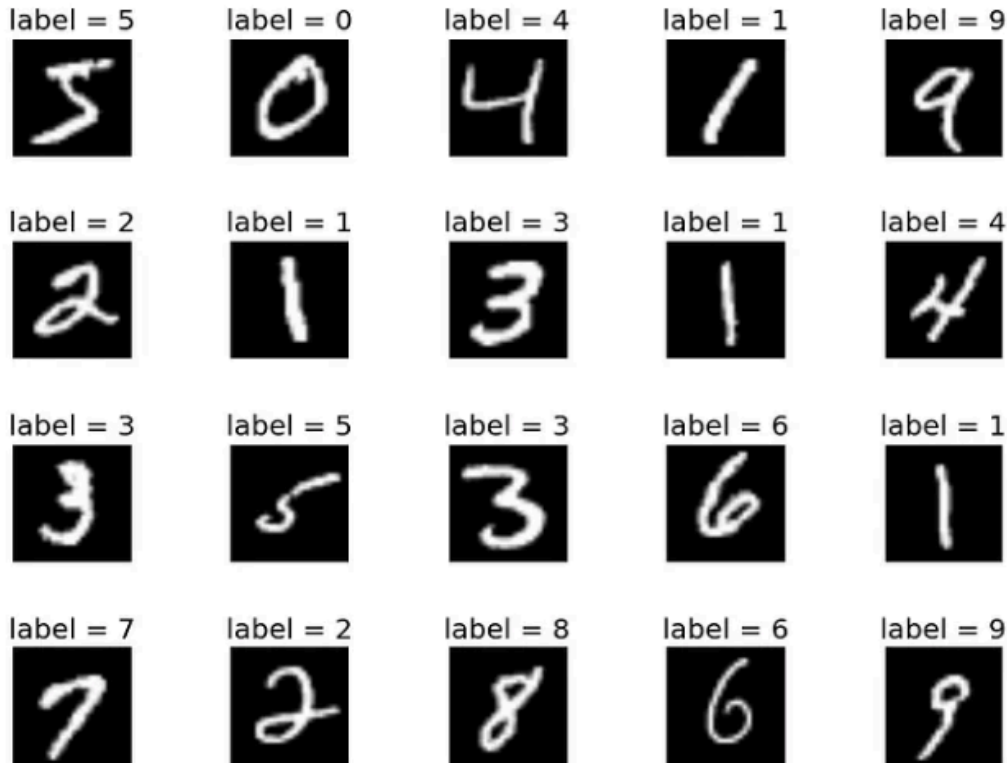
Génération de visages (du meme auteur) :

Dataset : CelebA



Notre implémentation :

Dataset : MNIST (60 000 images de chiffres manuscrits de 0 à 9)



MNIST, the "Hello World" of deep learning. © Yann LeCun et al.

Normalisation : Nous avons normalisé les images en ramenant les valeurs de pixels dans l'intervalle $[-1,1]$, en divisant par 127.5 puis en soustrayant 1, afin de faciliter l'apprentissage du réseau.

Au cours de notre expérimentation, nous avons développé deux versions de notre code, chacune avec des architectures et des hyperparamètres légèrement différents. Ces versions avaient pour but d'explorer différentes configurations possibles du GAN. Toutefois, en raison de certaines limitations techniques et d'un manque de temps, le modèle n'a pas totalement convergé. Dans les trois cas, nous avons observé que le discriminateur prenait systématiquement le dessus sur le générateur. Il est important de noter que le fait que la perte du générateur augmente ne signifie pas forcément qu'il régresse. En réalité, cela reflète souvent le fait que le discriminateur devient de plus en plus exigeant. Même lorsque le générateur s'améliore, il

continue à rencontrer des difficultés à le tromper. C'est le principe même des GANs : la progression du discriminateur pousse le générateur à s'améliorer jusqu'à, potentiellement, parvenir à produire des exemples indiscernables du réel.

Les deux implémentations :

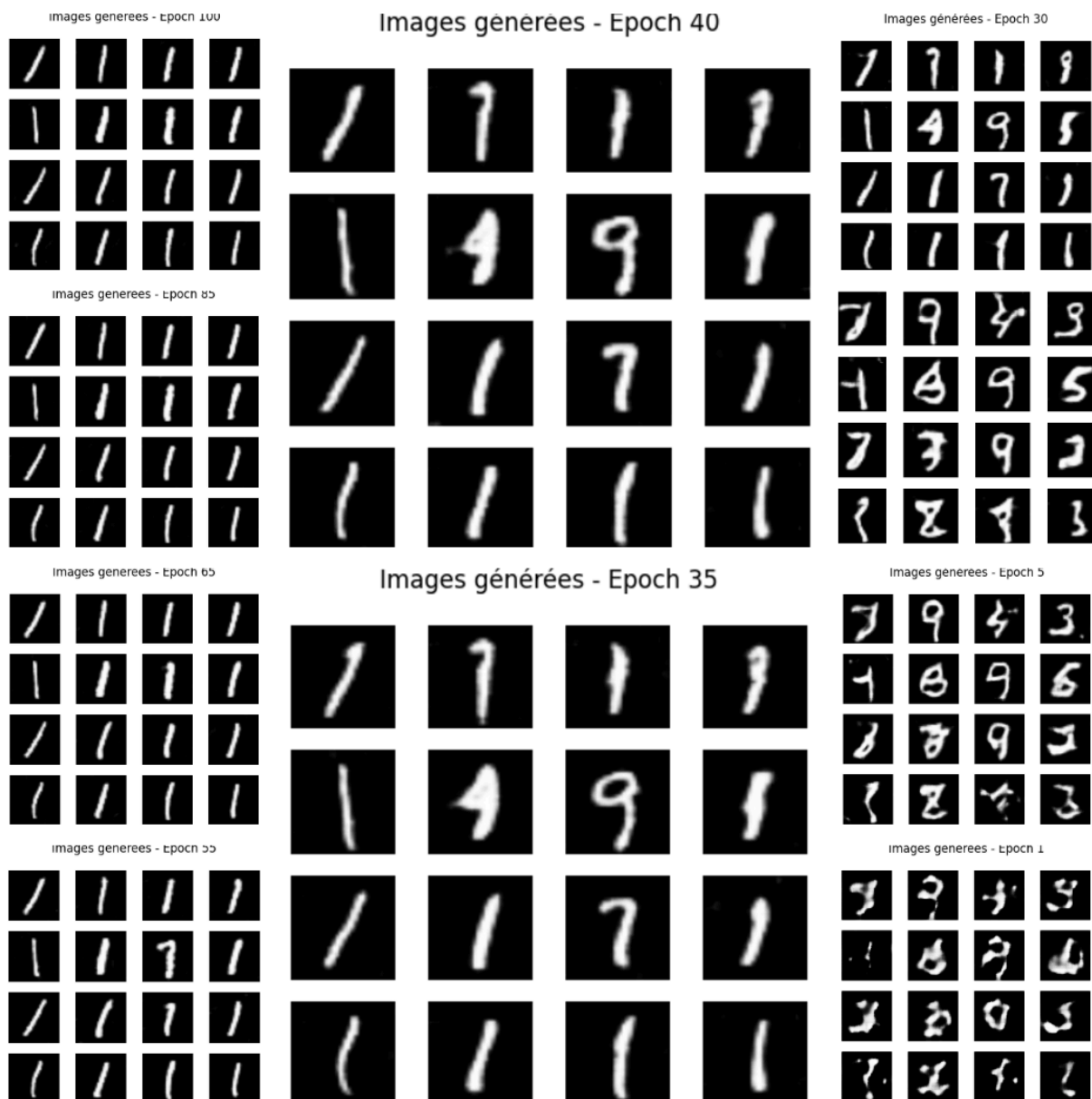
→ [Première implémentation](#)

→ [Deuxième implémentation](#)

Résultats obtenues dans la première version :

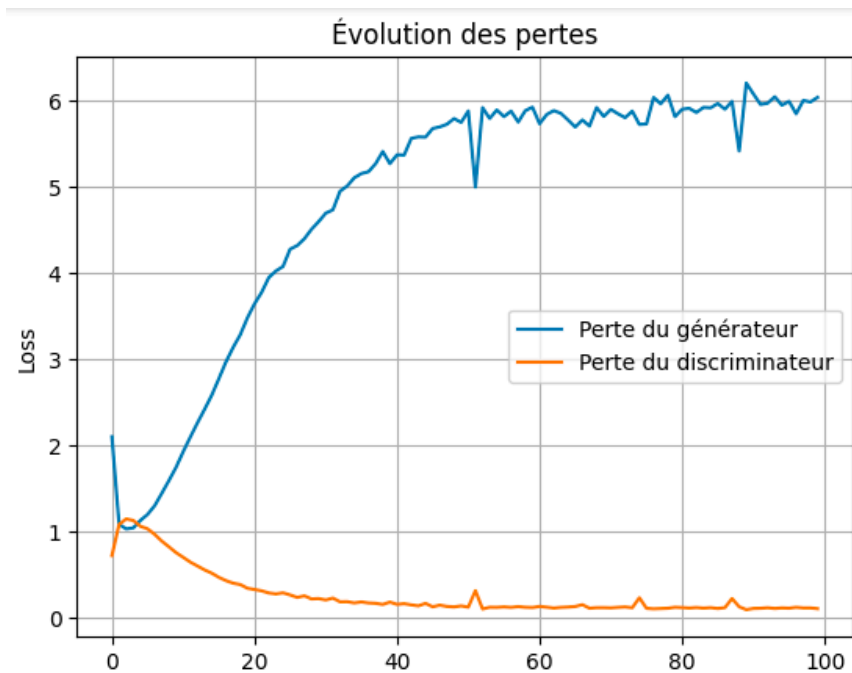
Quelques hyperparamètres

Taille des images	64 x 64
Taille du batch	64
Dimension du bruit	100
Nombre d'époques	100
Learning rate (Adam)	0.0002
Fonction de perte	BCE
Activation finale (D)	tanh
Conv2DTranspose	3 couches
Conv2D	3 couches



À partir de l'époque 40 jusqu'à la fin de l'entraînement (epoch 100), on observe que les images générées deviennent très similaires entre elles, ce qui suggère un phénomène de mode collapse. Cela signifie que le générateur se concentre sur une seule ou quelques sorties qui réussissent à tromper le discriminateur, au détriment de la diversité des échantillons

La courbe de perte :



Le générateur ici sur 100 epochs, déconverge, il est courant que ce phénomène se produise au début de l'entraînement, on a cependant pas essayé sur un nombre d'epochs conséquent pour voir cette courbe va descendre.

Résultats obtenues dans la deuxième version :

Nb : ici on génère les images avec la même seed à chaque epoch.

Epoch 1	Epoch 30	Epoch 60	Epoch 90
			
			
			
			
			
			
			
			
			
			
			
			
			
			
			
			

On observe ici presque les mêmes problèmes que dans la première implémentation.

VII.Conclusion :

L'exploration des Generative Adversarial Networks (GANs), et plus précisément des Deep Convolutional GANs (DCGANs), dans ce rapport, a permis de mettre en avant à la fois leur potentiel impressionnant pour générer des données réalistes et les nombreux défis liés à leur entraînement. L'apprentissage principal concerne la compréhension du jeu d'équilibre entre le générateur et le discriminateur, un mécanisme ingénieux qui, bien que complexe, est au cœur de la capacité des GANs à apprendre la distribution des données.

Notre tentative d'implémentation, bien qu'ayant rencontré des difficultés de convergence et l'apparition du mode collapse, a renforcé notre compréhension pratique de la sensibilité des GANs à la conception du modèle et aux choix des hyperparamètres. Le fait que le discriminateur prenne progressivement le dessus et les premiers signes de mode collapse observés sur le jeu de données MNIST montrent l'importance de bien configurer le modèle et d'adapter la durée de l'entraînement.

Cette expérience a été extrêmement enrichissante. Elle nous a permis de faire de nombreuses découvertes, d'approfondir notre compréhension des GANs et de mieux saisir les défis liés à leur entraînement. Travailler à deux nous a aussi aidés à échanger des idées, à résoudre des problèmes ensemble, et surtout à prendre énormément de plaisir tout au long du projet. Malgré les difficultés, nous en tirons un vrai sentiment de satisfaction et nous voyons déjà plein de pistes d'amélioration pour la suite, tel que :

- **Optimisation des hyperparamètres** : Ajuster le taux d'apprentissage, la taille du batch, et les optimisateurs pour trouver la meilleure configuration.
- **Augmentation des données** : Enrichir le jeu de données par des transformations pour améliorer la diversité et la qualité des images générées.
- **Entraînement prolongé et ajusté** : Ajuster les fréquences d'entraînement du générateur et du discriminateur pour éviter le déséquilibre.
- **Équilibrage générateur/discriminateur** : Maintenir un bon équilibre entre les deux réseaux pour éviter que l'un ne prenne le dessus.

VIII.BIBLIOGRAPHIE :

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). **Generative adversarial nets**. *Advances in Neural Information Processing Systems*, 27, 2672–2680.
2. Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., & Bharath, A. A. (2018). *Generative adversarial networks: An overview*. *IEEE Signal Processing Magazine*, 35(1), 53–65. <https://doi.org/10.1109/MSP.2017.2765202>
3. Grand View Research. (2024). *Generative Adversarial Networks Market Size, Share & Trends Analysis Report*. Récupéré de <https://www.grandviewresearch.com/industry-analysis/generative-adversarial-networks-market-report>
4. AmplifAI. (2024). *40 Generative AI Statistics You Need to Know in 2024*. Récupéré le 25 avril 2025 de <https://www.amplifai.com/blog/generative-ai-statistics#generative-ai-statistic-1>
5. Spectra-GANs. (2025). *Spectra-GANs: A new automated denoising method for low-S/N stellar spectra - Scientific figure*. Récupéré le 25 avril 2025, de https://www.researchgate.net/figure/GANs-structure-schematic_fig1_341953469
6. Saxena, D., & Cao, J. (2020). *Generative Adversarial Networks (GANs) Survey: Challenges, Solutions, and Future Directions*. arXiv preprint arXiv:2005.00065.
7. Clickworker. (2023). *Generative Adversarial Networks (GANs)*. <https://www.clickworker.com/ai-glossary/generative-adversarial-networks/>
8. Mustafa, M. (2022). *GANs Specialization Part 6: Interpretation and Disentanglement in GANs*. Medium. Retrieved from <https://medium.com/@Mustafa77/gans-specialization-part6-791e4e1358c5>
9. Tree Learning. (2024, 19 février). *Deep learning : des systèmes inspirés du cerveau humain*. Tree Learning. <https://www.tree-learning.fr/plateforme-lms-elearning/deep-learning/>